Lecture 10: Greedy Algorithms (Continued)

Huffman Coding · Set Cover Problem

1 Huffman Coding

1.1 The Coding Problem

Definition 1.1 (Coding Problem)**. Goal:** Represent symbols of a large alphabet (say $\Sigma = \{A, B, C, \ldots, Z\}$) with symbols from a smaller alphabet (say binary $\{0, 1\}$).

Requirements:

1. Encode/decode efficiently
2. Encoding must be unambiguous (not confusing)
3. Use few symbols (efficient in terms of space/bits used)

Example 1.1 (Motivation)**.** We want to represent an English novel in binary in as compact a form as possible.

How do we measure compactness?
Two possible objectives:

1. **Minimize maximum length of a codeword**
   Example: 5 digits to represent 26 letters ($\lceil \log_2 26 \rceil = 5$ bits)

2. **Minimize average length of a codeword**
   More useful if we have prior knowledge! Average by frequency instead of uniform distribution.

Idea: Use fewer binary digits to encode frequent letters like E, S, T, and more digits to encode rare letters like Z, Q, X.

1.2 Problem Formulation

Definition 1.2 (Optimal Prefix-Free Code Problem)**. Input:** Alphabet Σ of n symbols and frequencies (probabilities) f_i for $i \in \Sigma$ such that $f_i > 0$ and $\sum f_i = 1$.

Output: Distinct prefix-free binary codewords C_i for $i \in \Sigma$ that minimize the average codeword length according to frequencies.

The objective function is:

$$\sum_{i \in \Sigma} |C_i| \cdot f_i \quad \text{(average codeword length)}$$

Example: $A : 0001$, $B : 01$, $C : 1101$. We allow codewords of different lengths.

1.3 The Prefix-Free Requirement

Example 1.2 (Why we need prefix-free codes)**.** Consider a **bad encoding**:

$$A = 0, \quad B = 1, \quad C = 01$$

Problem: The bitstring 010101 could be decoded as:

- $ABABAB$ (interpreting as 0-1-0-1-0-1)
- CCC (interpreting as 01-01-01)
- $ABCAB$, etc.

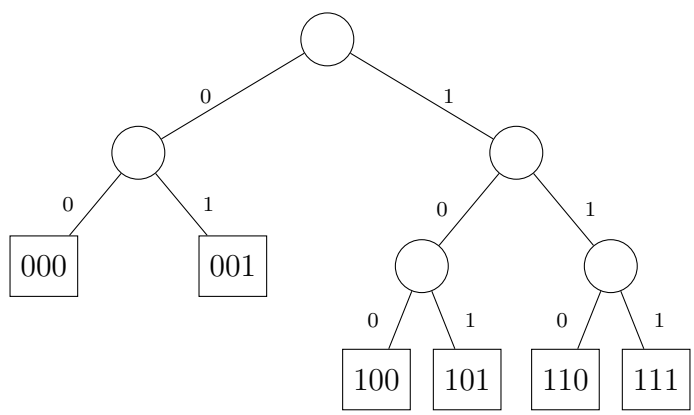
This is **ambiguous**! The problem is that 0 is a prefix of 01.

Definition 1.3 (Prefix-Free Code)**.** A code is **prefix-free** if C_i is not a prefix of C_j for any $i \neq j$.

1.4 Tree Representation

Remark 1.1. **Key idea:** Represent codewords in a binary tree:

- Use a **full binary tree** (every internal node has degree 2)
- Leaves are labeled by symbols
- Edges on path from root to leaf give the codeword (left = 0, right = 1)



Example binary tree. Leaves form a prefix-free code.

Claim 1.1. *Leaves of a full binary tree form a prefix-free code.*

Proof. If codeword C_i were a prefix of C_j, then the leaf for symbol i would be an ancestor of the leaf for symbol j. But in a tree, leaves cannot be ancestors of other leaves. □

Claim 1.2. *If we have internal nodes of degree 1, the encoding is not optimal — we can remove redundant bits.*

Therefor an optimal solution can be represented as the leaves of a full binary tree.

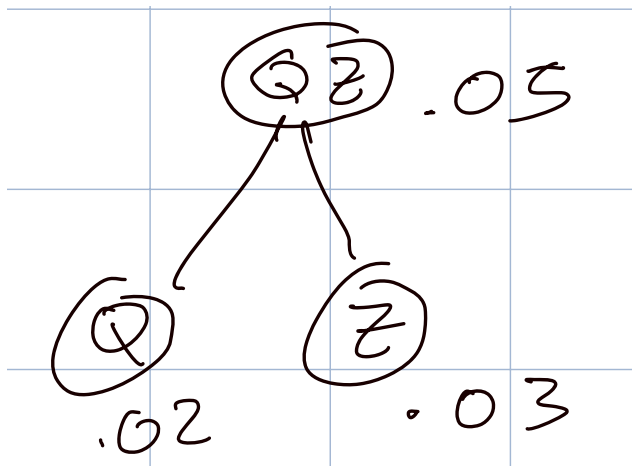


Figure 1: Basic Huffman operation: make two least frequent elements the children of the same node and combine

1.5 Reformulation as a Tree Problem

Definition 1.4 (Tree Formulation)**.** Given Σ and frequencies $\{f_i\}_{i\in\Sigma}$, construct a full binary tree with leaves labeled by distinct elements of Σ to minimize the **average depth of a leaf** (according to frequencies $\{f_i\}$).

The cost function is:

$$\text{Cost} = \sum_{i\in\Sigma} \text{depth}(i) \cdot f_i$$

1.6 Greedy Observations

Observation 1.1. We should put leaves with higher frequency at shallower depths (bottom layer).

Observation 1.2. We can permute leaves in any given layer without changing the average length.

Observation 1.3 (Key Greedy Insight). We can pick the two **least frequent** symbols and ensure they are on the bottom layer with a common parent.

Treat $\mathcal{QZ}$ as one symbol with combined frequency $0.02 + 0.03 = 0.05$ and recurse.

1.7 Huffman's Algorithm

Algorithm 1 Huffman's Algorithm

```
Initialize a priority queue Q with all symbols and their frequencies
while |Q| > 1 do
    i ← ExtractMin(Q)                                   ▷ Symbol with smallest frequency
    j ← ExtractMin(Q)                                   ▷ Second smallest
    Create new node z with left child i, right child j, and f_z = f_i + f_j
    Insert(Q, z)
end while
return root of tree
```

1.8 Example Execution

Example 1.3. Frequencies: $A : 0.3$, $B : 0.1$, $C : 0.2$, $D : 0.3$, $E : 0.1$

Step 1: Merge B and E (both 0.1):

A	BE	C	D
0.3	0.2	0.2	0.3

Step 2: Merge BE and C (both 0.2):

A	BEC	D
0.3	0.4	0.3

Step 3: Merge A and D (both 0.3):

AD	BEC
0.6	0.4

Step 4: Merge AD and BEC:

Resulting code:

Symbol	**Encoding**	**Size**
A	10	2
B	000	3
C	01	2
D	11	2
E	001	3

Average size:

$$0.3(2) + 0.1(3) + 0.2(2) + 0.3(2) + 0.1(3) = 0.6 + 0.3 + 0.4 + 0.6 + 0.3 = 2.2 \text{ bits/symbol}$$

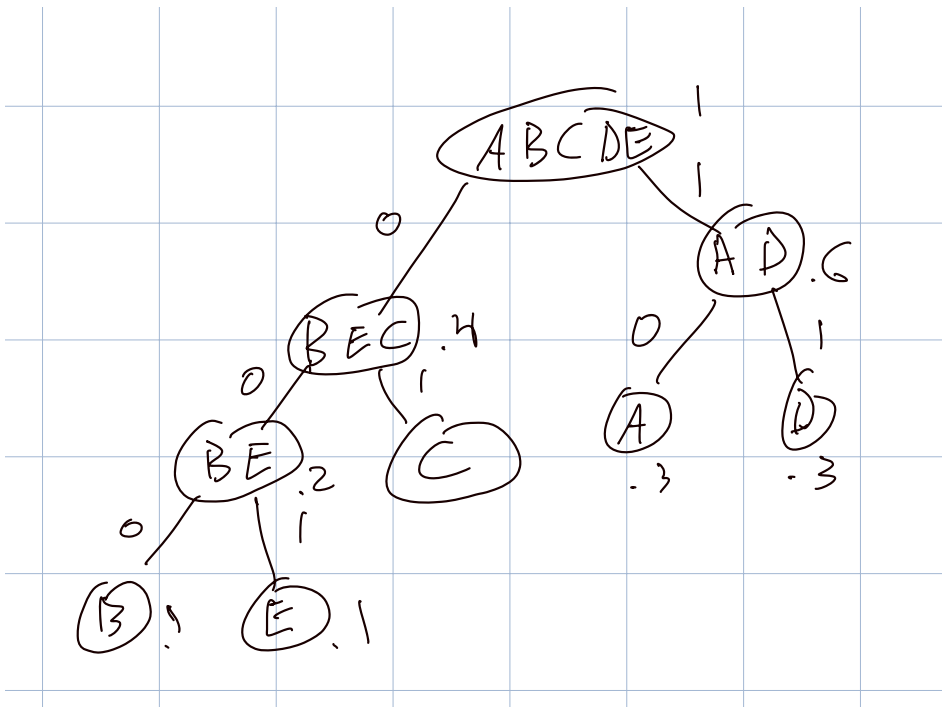


Figure 2: Huffman tree

2 Set Cover Problem

2.1 Problem Definition

Definition 2.1 (Set Cover Problem)**.** Given a set B of n elements (the universe) and a collection $\mathcal{X} = \{S_1, S_2, \ldots, S_m\}$ of subsets of B (where $S_i \subseteq B$) such that $\bigcup_{i=1}^{m} S_i = B$ (i.e., $\mathcal{X}$ covers B), find a subcollection $\mathcal{Y} \subseteq \mathcal{X}$ of smallest size such that $\bigcup_{S \in \mathcal{Y}} S = B$ (i.e., $\mathcal{Y}$ covers B).

Remark 2.1. **Complexity:** Set Cover is an **NP-hard** problem. We believe there is no polynomial-time algorithm to solve this problem optimally.

2.2 Greedy Algorithm

Since we cannot find the optimal solution efficiently, we analyze the performance of a greedy algorithm.

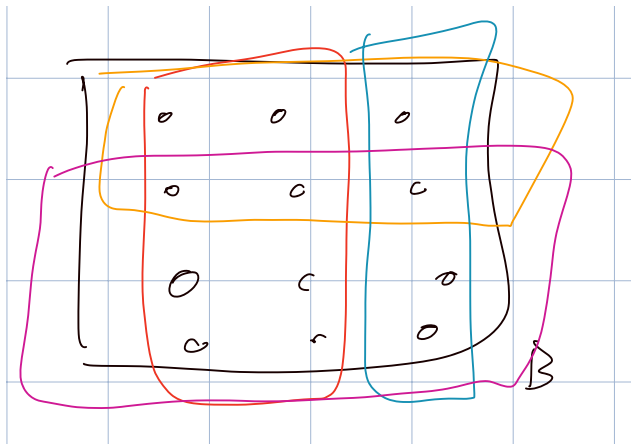


Figure 3: Set cover instance. Minimum cover is size 2 (eg Pink and Orange)

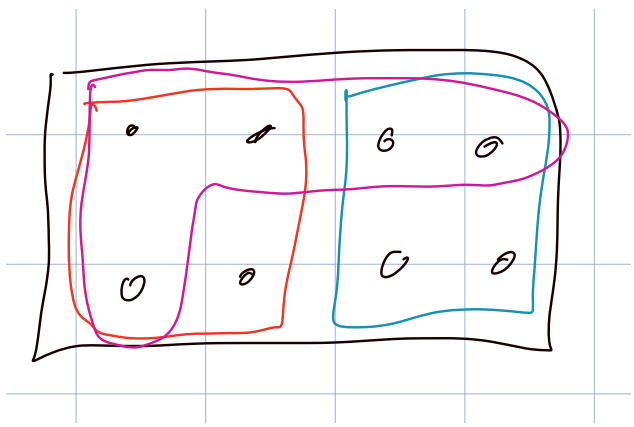


Figure 4: Greedy picks 3 sets; OPT picks 2

Algorithm 2 Greedy Set Cover

```
𝒴 ← ∅                                          ▷ Solution
R ← B                                          ▷ Remaining uncovered elements
while R ≠ ∅ do
    Choose S ∈ 𝒳 that covers the most currently uncovered elements
    𝒴 ← 𝒴 ∪ {S}
    R ← R \ S
end while
return 𝒴
```

Key idea: At each step, add the set that covers the most currently uncovered elements. Simple and easy to implement!

2.3 Example Execution

Example 2.1. See Figure 4: **Optimal:** Use 2 sets (e.g., Red and Blue cover all 8 elements)

Greedy: Will choose all 3 sets

How good was this solution?

$$\text{Ratio} = \frac{\text{Greedy}}{\text{OPT}} = \frac{3}{2}$$

In general, what approximation ratio can we guarantee for greedy?

2.4 Approximation Guarantee

Theorem 2.1 (Greedy Approximation for Set Cover)**.** *The size of a set cover returned by the greedy algorithm is at most* $\ln n \cdot OPT$*, where* $n = |B|$ *is the size of the universe.*

Proof. Let $k = \text{OPT}$ (size of optimal solution).

Let B_i be the set of uncovered elements after step i of greedy.

Initial condition: $|B_0| = n$

If $|B_t| < 1$, we are done, all elements are covered.

Claim 2.1. *The optimal solution uses* k *sets to cover* B_{i-1}*, so there is some set* S *that covers* $\geq \frac{|B_{i-1}|}{k}$ *elements of* B_{i-1}*.*

Since greedy picks the set covering the most uncovered elements:

$$|B_i| \leq |B_{i-1}| - \frac{|B_{i-1}|}{k} = \left(1 - \frac{1}{k}\right)|B_{i-1}|$$

By induction:

$$\begin{aligned}
|B_0| &= n \\
|B_1| &\leq \left(1 - \frac{1}{k}\right) n \\
|B_2| &\leq \left(1 - \frac{1}{k}\right)^2 n \\
|B_3| &\leq \left(1 - \frac{1}{k}\right)^3 n \\
&\vdots \\
|B_t| &\leq \left(1 - \frac{1}{k}\right)^t n
\end{aligned}$$

Using the inequality $1 - x < e^{-x}$ for $x > 0$:

$$|B_t| < e^{-t/k} \cdot n$$

We stop when $|B_t| < 1$, and if $t \geq k \ln n$ then $e^{-t/k} \cdot n < 1$ and so $|B_t| < 1$.
Therefore, greedy uses at most $t \leq k \ln n = \text{OPT} \cdot \ln n$ sets. □ □

Interpretation:

$$\frac{\text{Greedy}}{\text{OPT}} \leq \ln n$$

This means greedy is an $\ln n$-approximation algorithm.